

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И
ОПТИКИ

ОТЧЁТ

по дисциплине **«Архитектура игровых движков»**
«WYSIWYG редактор сцены на базе Dear ImGui»

Студент: Тереховский Арсений Дмитриевич
Группа: J3300

Санкт-Петербург
2026 г.

Содержание

1	Введение	4
2	Постановка задачи	4
2.1	Цель и обязательные задачи	4
2.2	Расширение задания в проекте	5
3	Архитектура решения	6
3.1	Интеграция ImGui в игровой цикл	6
3.2	Связь редактора и ECS	7
3.3	Подсистема layout и docking	8
4	Реализация ключевых модулей	9
4.1	Подключение ImGui и ImGuizmo	9
4.2	Главное меню, toolbar и стиль редактора	9
4.3	Панель иерархии сцены	10
4.4	Выбор объекта кликом в сцене	10
4.5	Инспектор компонентов	11
4.6	Viewport и live preview	12
4.7	Интеграция ImGuizmo	13
4.8	Edit Mode и Play Mode	14
4.9	Панель статистики	15
4.10	Asset Browser и превью ресурсов	15
4.11	Сохранение сцены и Undo/Redo	16
4.12	Docking, resize и сохранение layout	16
5	Результаты	18
5.1	Функциональный результат	18
5.2	Сопоставление с требованиями задания	19
5.3	Дополнительные результаты	19
5.4	Иллюстрации и проверка работоспособности	20
6	Возникшие трудности и способы их решения	20
7	Ответы на контрольные вопросы	22

1. Введение

Практическое занятие №5 посвящено созданию WYSIWYG-редактора сцены для существующего игрового движка. Цель работы состояла в том, чтобы интегрировать в движок библиотеку Dear ImGui, построить редакторский интерфейс, связать его с ECS, обеспечить редактирование компонентов в реальном времени, добавить 3D-манипуляторы трансформаций и реализовать переключение между режимами редактирования и игры.

В текущей версии *TurboGameEngine* редактор реализован как полноценный слой инструментов поверх уже существующего рантайма. Он использует ECS-мир, физическую систему, рендерер на базе Diligent Engine, подсистему ввода и сцену из предыдущих практических работ. Новая работа не ограничилась минимальной тестовой отрисовкой ImGui: в проект добавлены инспектор компонентов, иерархия сцены, viewport с live preview, ImGuizmo, статистика, Asset Browser, сохранение и загрузка сцены, Undo/Redo, выбор объекта кликом в 3D-сцене, профессиональная тёмная тема, drag-and-drop docking, resize закреплённых панелей и сохранение раскладки между запусками. После пользовательского тестирования также были исправлены скачки камеры, визуальные артефакты docking layout, assert ImGui в окне viewport и нестабильное вращение/масштабирование через гизмо.

2. Постановка задачи

2.1. Цель и обязательные задачи

Согласно заданию, необходимо было выполнить следующие пункты:

1. интегрировать Dear ImGui в существующий движок;
2. создать главное меню и базовые панели инструментов;
3. реализовать панель инспектора для просмотра и редактирования компонентов выбранной сущности;
4. разработать панель иерархии сцены;
5. обеспечить live preview, при котором изменения в инспекторе сразу видны на экране;

6. внедрить 3D-гизмо для перемещения, вращения и масштабирования объектов;
7. создать панель статистики с FPS, временем кадра и количеством объектов;
8. обеспечить переключение между Edit Mode и Play Mode.

2.2. Расширение задания в проекте

Помимо обязательных пунктов, в проект были добавлены дополнительные функции, которые приближают редактор к инструментам уровня Unity, Unreal Engine и Roblox Studio:

- профессиональная раскладка окон с главным меню, toolbar, статусной строкой, viewport, hierarchy, inspector, statistics и content browser;
- кастомная docking-система с drag-and-drop переносом панелей между зонами;
- resize закреплённых областей через splitters внутри docking layout;
- автоматическое сохранение layout в `assets/editor_layout.json`;
- выбор объектов кликом прямо в сцене с корректным ray picking ближайшего объекта;
- исключение editor-camera из начального выбора и из picking-логики;
- Undo/Redo для действий редактора и операций гизмо;
- сохранение и загрузка сцены в JSON;
- Asset Browser с карточками, фильтром, metadata и превью ресурсов;
- фиксированный центральный viewport, который не отцепляется и не замещается другими панелями;
- защита free camera от резких скачков мыши и больших dt-спайков;

- стабильное применение rotate/scale в ImGuizmo без паразитного изменения соседних компонентов transform;
- оптимизации UI и picking: кэширование ассетов, throttling hover-pick, отключение лишней grid-сетки, уменьшение затрат на отрисовку редакторских overlays.

3. Архитектура решения

3.1. Интеграция ImGui в игровой цикл

Интерфейс редактора встроен в обычный игровой цикл движка. Платформенный слой создаёт окно и передаёт события ввода, затем состояние игры обновляет ECS и физику, рендерер отрисовывает 3D-сцену, а поверх неё вызывается редакторский UI. Такой порядок важен: ImGui должен быть отрисован последним, чтобы панели и гизмо находились поверх viewport.

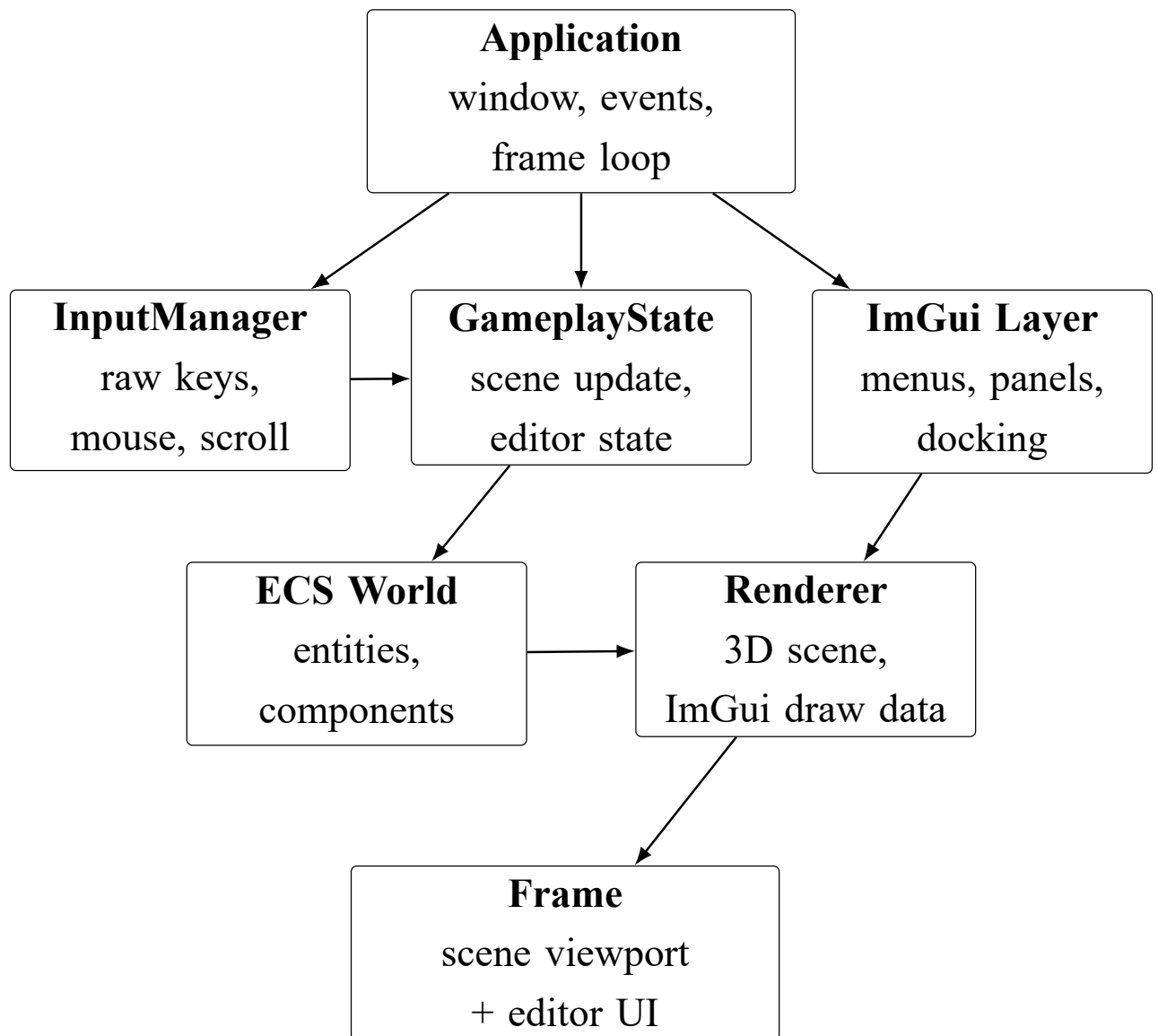


Рис. 1 – Место редакторского UI в игровом цикле

3.2. Связь редактора и ECS

Редактор не хранит отдельную копию сцены для отображения. Он напрямую работает с ECS-компонентами выбранной сущности. За счёт этого live preview получается естественным: инспектор изменяет Transform, MeshRenderer, Rigidbody, Collider и другие компоненты, а рендерер уже в следующем кадре видит актуальные значения.

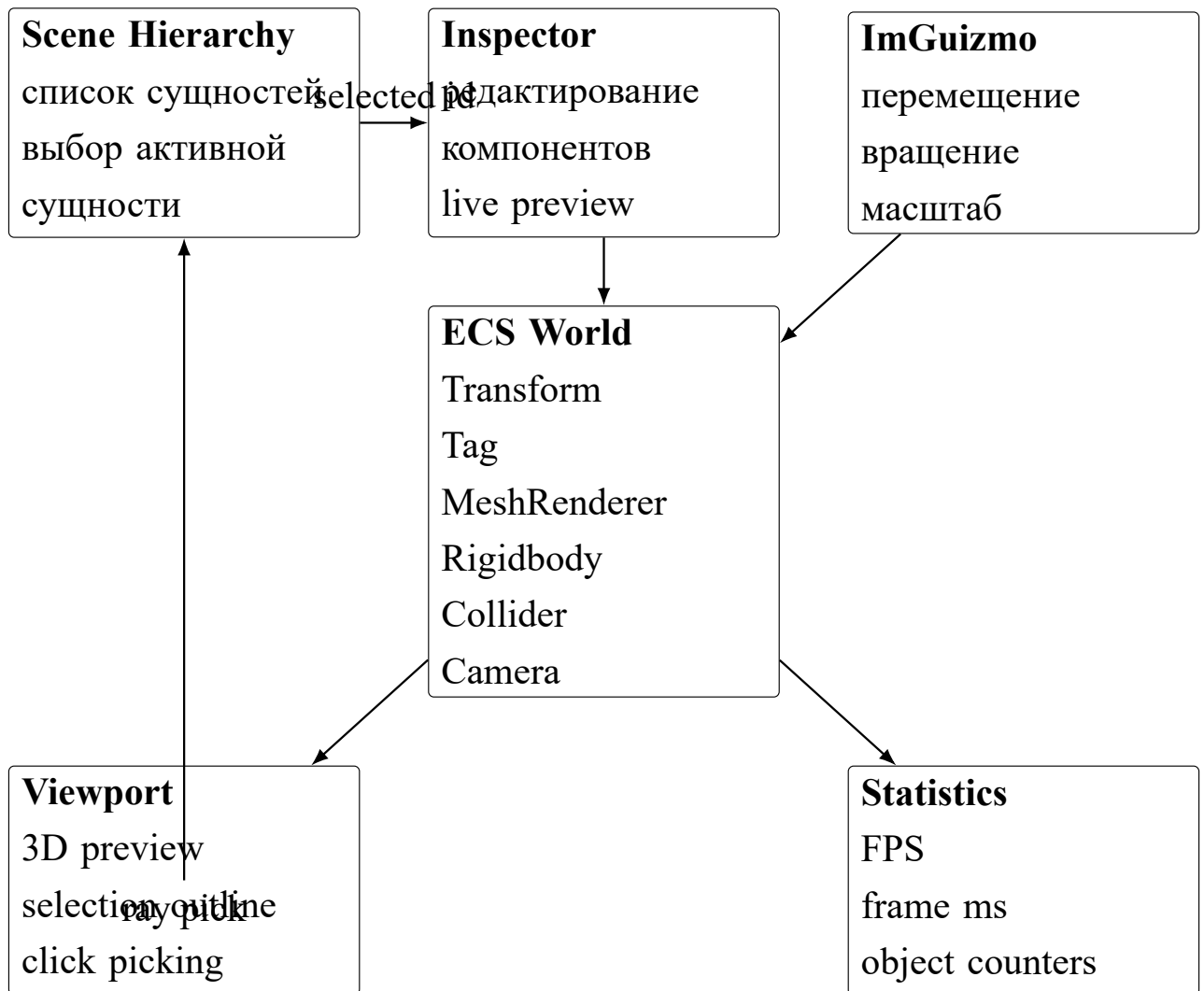


Рис. 2 – Взаимодействие панелей редактора с ECS

3.3. Подсистема layout и docking

В поставленном задании рекомендована docking-ветка ImGui. В проекте используется версия ImGui, поставляемая вместе с Diligent Tools, в которой отсутствует полноценный DockSpace. Поэтому была реализована собственная docking-модель поверх базовых окон ImGui:

- каждая панель имеет логический слот Top, Center, LeftTop, LeftBottom, RightTop, RightBottom, Bottom или Floating;
- у панели есть drag-grip; при перетаскивании открывается fullscreen overlay с зонами docking;
- drop в зону закрепляет панель, drop вне зон переводит её в floating;
- если целевой слот занят, панели меняются местами;

- закреплённые панели имеют `resize` через `splitters`;
- центральный слот зарезервирован под `viewport` и не принимает другие панели;
- служебные `resize-hitbox` не рисуют постоянные рамки и показывают подсветку только при наведении;
- `layout` сохраняется между запусками в JSON.

4. Реализация ключевых модулей

4.1. Подключение ImGui и ImGuizmo

Интеграция редакторского UI начинается с настройки сборки. В проект подключены заголовки и библиотека ImGui, уже входящие в Diligent Tools, а также ImGuizmo для 3D-манипуляторов. Для корректной сборки ImGuizmo добавлен `compatibility header`, подключаемый через `forced include`, так как библиотека ожидает некоторые `math`-операторы ImGui.

Листинг 1 – Фрагмент подключения ImGuizmo и ImGui в CMake

```

1 FetchContent_Declare(
2   imguizmo
3   GIT_REPOSITORY https://github.com/CedricGuillemet/ImGuizmo.git
4   GIT_TAG 1.83
5 )
6
7 add_library(imguizmo STATIC ${imguizmo_SOURCE_DIR}/ImGuizmo.cpp)
8 target_include_directories(imguizmo PUBLIC ${imguizmo_SOURCE_DIR})
9 target_compile_definitions(imguizmo PUBLIC IMGUI_DEFINE_MATH_OPERATORS)
10
11 target_link_libraries(engine_game PRIVATE Diligent-ImGui imguizmo)
12 target_compile_options(engine_game PRIVATE /FI"engine/game/ImGuizmoCompat.h")

```

4.2. Главное меню, toolbar и стиль редактора

В редакторе создана единая тёмная тема, близкая к профессиональным игровым редакторам. Основные действия доступны через меню `File`, `Edit`, `GameObject`, `Tools`, `Scene`, `View`, `Window`, `Help`. `Toolbar` содержит кнопки

Edit, Play, Undo, Redo, создание куба, дублирование, фокус камеры, режимы гизмо, локальное/глобальное пространство, snap и скорость камеры.

Для удобства добавлена статусная строка. Она показывает текущий режим, выбранную сущность, скорость камеры, размер undo/redo history и последнее сообщение редактора. Это упрощает диагностику действий пользователя.

4.3. Панель иерархии сцены

Панель Scene Hierarchy выводит список всех живых сущностей ECS. Для отображения используется имя из компонента Tag; если тега нет, генерируется имя по идентификатору. Панель поддерживает фильтрацию, выбор сущности, создание объектов, дублирование и удаление.

Выбор через иерархию синхронизирован с инспектором, viewport и гизмо. Это означает, что активная сущность всегда одна: её свойства отображаются справа, вокруг неё рисуется selection outline, а ImGuizmo получает именно её transform matrix.

4.4. Выбор объекта кликом в сцене

Дополнительно к выбору из списка реализован picking прямо во viewport. При клике мыши строится луч из камеры через позицию курсора в 3D-пространство. Далее перебираются потенциально выбираемые сущности, для каждой вычисляется pick-shape, после чего выбирается ближайшее пересечение. Это решает проблему, когда объект на заднем плане выбирался вместо ближайшего куба.

Листинг 2 – Упрощённая схема выбора ближайшей сущности лучом

```
1 std::optional<PickResult> GameplayState::pickEntity(const PickRay& ray) const
2 {
3     PickResult best{};
4     best.distance = std::numeric_limits<float>::max();
5
6     m_world.forEachEntity([&](ecs::EntityId entity) {
7         if (entity == m_cameraEntity) {
8             return;
9         }
10
11         ecs::WorldColliderShape shape{};
12         if (!computeEntityPickShape(entity, shape)) {
```

```

12         return;
13     }
14
15     float distance = 0.0F;
16     const bool hit = shape.type == ecs::ColliderType::Sphere
17         ? intersectRaySphere(ray.origin, ray.direction, shape.sphere,
18             distance)
19         : intersectRayAabb(ray.origin, ray.direction, shape.aabb, distance
20             );
21
22     if (hit && distance < best.distance) {
23         best.entity = entity;
24         best.distance = distance;
25     }
26
27     });
28
29     return best.entity != ecs::kInvalidEntity ? std::optional(best) : std::
30         nullopt;
31 }

```

4.5. Инспектор компонентов

Панель Inspector отображает компоненты выбранной сущности. Для каждого компонента создан отдельный блок. Сейчас редактируются:

- Tag: имя объекта;
- Transform: позиция, поворот и масштаб;
- MeshRenderer: видимость, tint, UV scale, primitive type, mesh, texture, shader;
- Rigidbody: gravity, static, kinematic, масса, трение, restitution, velocity;
- Collider: параметры AABB и Sphere;
- Camera: параметры камеры, если компонент присутствует.

Листинг 3 – Пример редактирования Transform через ImGui

```

1 if (auto* transform = m_world.getComponent<ecs::Transform>(entity)) {
2     if (ImGui::CollapsingHeader("Transform", ImGuiTreeNodeFlags_DefaultOpen))
3     {
4         if (drawVec3Editor("Position", transform->position, 0.05F, "%.2f")) {
5             wakeEntity(entity);
6         }
7     }
8 }

```

```

5         }
6
7         glm::vec3 rotationDegrees = glm::degrees(transform->
            rotationEulerRadians);
8         if (drawVec3Editor("Rotation", rotationDegrees, 0.25F, "%.1f deg")) {
9             transform->rotationEulerRadians = glm::radians(rotationDegrees);
10            wakeEntity(entity);
11        }
12
13        if (drawVec3Editor("Scale", transform->scale, 0.03F, "%.2f")) {
14            transform->scale = glm::max(transform->scale, glm::vec3(0.05F));
15            wakeEntity(entity);
16        }
17    }
18 }

```

Все изменения выполняются непосредственно над ECS-компонентом, поэтому результат сразу виден в viewport. Для редактирования в Edit Mode система пишет snapshots в историю, что позволяет отменять действия через Undo/Redo.

4.6. Viewport и live preview

Viewport занимает центральную область редактора. В отличие от остальных панелей он закреплён в центре и не участвует в drag-and-drop перестановке: это предотвращает случайное разрушение основной рабочей области. Сцена отрисовывается обычным renderer pipeline, но поверх неё добавляются редакторские overlay-элементы: подсказки режима, selection outline, hover outline и ImGuizmo. При изменении свойства в инспекторе сцена не перезапускается и не пересоздаётся. Изменяется компонент в ECS, затем следующий кадр рисуется уже по новым данным.

Для удобства добавлены команды:

- F: фокус камеры на выбранном объекте;
- Frame Scene: кадрирование всей сцены;
- camera bookmarks;
- изменение скорости камеры колесом мыши;
- free camera через RMB + WASDQE.

Для устойчивости редакторской камеры добавлены два защитных механизма. При включении и выключении camera-look сбрасывается накопленная mouse delta, чтобы переключение режима курсора не превращалось в резкий рывок камеры. Кроме того, контроллер камеры ограничивает максимальный raw mouse delta и максимальный dt, поэтому после лагов, breakpoint или потери фокуса камера не улетает далеко от сцены.

4.7. Интеграция ImGuizmo

Гизмо работает поверх viewport. Для выбранной сущности строится матрица transform, затем в ImGuizmo::Manipulate() передаются view/projection матрицы активной камеры, режим операции и система координат. После тестирования была изменена схема применения результата: редактор больше не записывает в Transform все три компонента сразу после каждой операции. Это важно, потому что универсальное разложение матрицы на Euler-углы и scale может давать численный шум. Теперь каждая операция меняет только релевантную часть transform: Move меняет позицию, Rotate меняет только rotation, а Scale меняет только scale.

Листинг 4 – Принцип работы ImGuizmo в viewport

```
1  ImGuizmo::SetOrthographic(false);
2  ImGuizmo::SetDrawlist();
3  ImGuizmo::SetRect(
4      m_viewportPosition.x,
5      m_viewportPosition.y,
6      m_viewportSize.x,
7      m_viewportSize.y);
8
9  glm::mat4 localMatrix = ecs::computeWorldMatrix(m_world, m_selectedEntity);
10 glm::mat4 deltaMatrix(1.0F);
11 const bool manipulated = ImGuizmo::Manipulate(
12     glm::value_ptr(cameraFrame.view),
13     glm::value_ptr(cameraFrame.projection),
14     operation,
15     mode,
16     glm::value_ptr(localMatrix),
17     glm::value_ptr(deltaMatrix),
18     m_snapEnabled ? snapValues : nullptr);
19
20 if (manipulated) {
21     auto components = decomposeEditorTransform(
22         localMatrix,
```

```

23         transform->rotationEulerRadians,
24         transform->scale);
25
26     if (components.has_value()) {
27         transform->position = components->position;
28
29         switch (m_gizmoOperation) {
30             case GizmoOperation::Translate:
31                 break;
32             case GizmoOperation::Rotate:
33                 transform->rotationEulerRadians = components->
34                     rotationEulerRadians;
35                 break;
36             case GizmoOperation::Scale:
37                 transform->scale = components->scale;
38                 break;
39         }
40     }

```

Вспомогательная функция `decomposeEditorTransform()` проверяет конечность матрицы, ограничивает `scale` безопасным диапазоном, ортонормализует оси и разворачивает Euler-углы относительно предыдущего значения. За счёт этого при вращении объект не начинает менять масштаб, а при масштабировании не появляется паразитное вращение. Если у объекта есть `Rigidbody`, во время прямого редактирования дополнительно обнуляются `linear/angular velocity` и накопленные силы, чтобы физика не продолжала крутить объект поверх действия пользователя.

Toolbar позволяет переключать операции `Move`, `Rotate`, `Scale`, выбирать локальную или глобальную систему координат, а также включать `snap`. Изменения через гизмо сохраняются в историю как отдельное действие.

4.8. Edit Mode и Play Mode

В режиме `Edit` физическая симуляция остановлена, а инспектор и гизмо доступны для редактирования. При переходе в `Play` создаётся `snapshot` текущей сцены, включается физика и игровая логика. При остановке `Play Mode` `snapshot` восстанавливается, поэтому тестирование не разрушает сцену редактора.

Это поведение соответствует `workflow` профессиональных редакторов:

пользователь может настроить сцену, нажать Play, проверить физику и затем вернуться к исходному состоянию.

4.9. Панель статистики

Панель `Statistics` отображает:

- FPS;
- время кадра в миллисекундах;
- CPU-время отрисовки сцены;
- графики `frame ms` и `render ms`;
- число сущностей;
- число `renderable` объектов;
- число камер;
- число динамических тел;
- `collision counters`.

История значений хранится в кольцевых массивах, что даёт недорогие графики без постоянных аллокаций.

4.10. Asset Browser и превью ресурсов

В рамках самостоятельного расширения задания реализована панель `Content Browser`. Она сканирует папку `assets`, классифицирует ресурсы и показывает их не текстовым списком, а карточками, похожими на `content browser` в профессиональных редакторах.

Для ассетов реализованы:

- фильтр по имени, типу и пути;
- карточки ресурсов с цветовой категоризацией;
- `metadata`: тип, размер файла, число вершин/граней для OBJ;

- pixel-preview для PPM-текстур;
- компактное wire-cube preview для mesh-файлов;
- кнопка применения mesh/texture/shader к выбранному объекту;
- загрузка сцены из `assets/scenes/editor_scene.json`.

4.11. Сохранение сцены и Undo/Redo

Меню `File` содержит `Save Scene` и `Load Scene`. Сцена сериализуется в JSON, включая основные ECS-компоненты и служебные данные редактора. Это продолжает работу предыдущей практической работы по сериализации.

Undo/Redo реализован через snapshots сцены. Перед редактированием создаётся состояние ``до'', а после подтверждения изменения оно помещается в `undo stack`. Такой подход прост, но надёжен для учебного проекта: он покрывает изменения в инспекторе, операции создания/удаления/дублирования сущностей и трансформации через гизмо.

4.12. Docking, resize и сохранение layout

После первичной реализации кнопочного dock-menu была добавлена более удобная модель в стиле Roblox Studio: пользователь зажимает `grip` панели и переносит её в подсвеченную зону. Для этого используется стандартный Drag and Drop API ImGui: `SetDragDropPayload()`, `AcceptDragDropPayload()` и отдельное overlay-окно с drop-зонами.

В последней версии также реализовано изменение размеров закреплённых областей. `Resize` работает через невидимые hitbox-окна ImGui, расположенные на границах docking layout:

- между левой колонкой и viewport;
- между viewport и правой колонкой;
- между `Scene Hierarchy` и `Content Browser`;
- между `Inspector` и `Statistics`.

После визуального тестирования `splitters` были доработаны: у служебных окон отключены `background` и `border`, поэтому они не оставляют на экране лишние тонкие линии. В обычном состоянии они невидимы, но при наведении показывают аккуратную подсветку и меняют курсор на `resize`. Также был исправлен `debug assert ImGui`, возникавший из-за `SetCursorScreenPos()` в окне `viewport` без последующего `item`: лишний вызов был удалён.

Размеры колонок и пропорции вертикальных стеков сохраняются в `assets/editor_layout.json`. При следующем запуске редактор восстанавливает `layout` автоматически.

Листинг 5 – Структура файла сохранения `layout`

```
1 {
2   "version": 1,
3   "locked": false,
4   "visibility": {
5     "toolbar": true,
6     "hierarchy": true,
7     "project": true,
8     "inspector": true,
9     "stats": true,
10    "viewport": true,
11    "status": true
12  },
13  "dockSlots": {
14    "toolbar": "top",
15    "hierarchy": "left_top",
16    "project": "left_bottom",
17    "inspector": "right_top",
18    "stats": "right_bottom",
19    "viewport": "center",
20    "status": "bottom"
21  },
22  "sizing": {
23    "leftWidth": 300.0,
24    "rightWidth": 390.0,
25    "leftSplit": 0.72,
26    "rightSplit": 0.68
27  }
28 }
```

5. Результаты

5.1. Функциональный результат

В результате работы движок получил полноценный WYSIWYG-редактор сцены. Теперь можно:

- открывать сцену в редакторском интерфейсе;
- выбирать объекты из иерархии или кликом во viewport;
- менять компоненты выбранной сущности через инспектор;
- сразу видеть изменения в 3D-сцене;
- перемещать, вращать и масштабировать объекты через ImGuizmo;
- тестировать сцену в Play Mode и возвращаться к исходному Edit Mode;
- сохранять и загружать сцену;
- пользоваться Undo/Redo;
- просматривать ассеты как карточки с превью;
- перемещать, закреплять и ресайзить панели редактора;
- сохранять центральный viewport стабильным и не допускать его случайного перемещения;
- пользоваться более устойчивой камерой без резких скачков после смены режима курсора;
- вращать и масштабировать объекты через гизмо без дрожания и паразитного изменения transform;
- сохранять layout между запусками.

5.2. Сопоставление с требованиями задания

№	Требование	Реализация в проекте
1	Интеграция Dear ImGui	Подключён ImGui слой через Diligent Tools, добавлен UI lifecycle и взаимодействие с вводом.
2	Главное меню и панели	Реализованы меню, toolbar, viewport, hierarchy, inspector, statistics, content browser и status bar.
3	Inspector	Редактируются Transform, Tag, MeshRenderer, Rigidbody, Collider, Camera и связанные параметры.
4	Scene Hierarchy	Отображается список сущностей, фильтр, выбор, создание, дублирование и удаление.
5	Live preview	Инспектор и гизмо меняют ECS-компоненты напрямую, viewport обновляется каждый кадр.
6	Гизмо	Подключён ImGuizmo: move, rotate, scale, local/global, snap, undo integration.
7	Statistics	FPS, frame ms, render ms, графики, количество сущностей, renderable объектов, камер и динамических тел.
8	Edit/Play	Play создаёт snapshot и включает физику; Stop восстанавливает сцену редактора.

Таблица 1 – Соответствие результата требованиям ПЗ5

5.3. Дополнительные результаты

Часть пунктов из самостоятельной работы также реализована:

- **Save/Load:** сцена сохраняется в JSON и загружается из редактора;
- **Undo/Redo:** реализована история действий редактора;
- **Asset Browser:** добавлена панель ассетов с визуальными карточками;
- **Кастомная тема:** интерфейс приведён к единому тёмному стилю;
- **Docking UX:** реализованы drag-and-drop docking, resize и сохранение layout;

- **Стабилизация редактора:** исправлены скачки камеры, визуальные артефакты splitters, assert ImGui в viewport и нестабильный rotate/scale.

5.4. Иллюстрации и проверка работоспособности

В текстовой версии отчёта основной упор сделан на архитектуру, листинги и описание результата. Для финальной печатной версии можно добавить актуальные скриншоты:

- общий вид редактора с hierarchy, viewport, inspector, statistics и content browser;
- выбранный объект с активным ImGuizmo;
- пример asset browser с карточками и превью ресурсов;
- пример перетаскивания панели на docking overlay;
- пример изменённого layout после resize splitters;
- пример стабильной работы rotate/scale gizmo на выбранном объекте.

6. Возникшие трудности и способы их решения

- **Отсутствие полноценной docking-ветки ImGui в текущей поставке.** В Diligent Tools используется ImGui без DockSpace. Решение: реализовать собственные dock slots, drag-and-drop overlay и сохранение layout поверх базовых окон ImGui.
- **Некорректный picking при перекрытии объектов.** Ранний выбор мог попадать в объект на заднем плане. Решение: выбирать ближайшее пересечение ray-shape и исключить editor camera из pickable объектов.
- **Конфликт горячих клавиш с ImGui.** Комбинации Ctrl+S, Ctrl+Z, Ctrl+Y не всегда проходили через обычный путь ввода. Решение: добавить raw input path и учитывать capture-флаги ImGui.

- **Скачки свободной камеры.** При смене режима курсора или после потери фокуса накапливалась большая mouse delta, из-за чего камера могла резко повернуться и пользователь терял сцену. Решение: сбрасывать mouse delta при включении/выключении camera-look, ограничить максимальную mouse delta и clamp dt в контроллере камеры.
- **Просадка FPS из-за лишних overlay-элементов.** Debug grid и частые проверки hover-pick создавали лишнюю нагрузку. Решение: отключить ненужную grid-сетку, кэшировать ассеты, ограничить частоту hover picking и уменьшить объём draw calls UI.
- **Странный вид mesh-preview в Asset Browser.** Первичная пиктограмма меша выглядела как длинный треугольник. Решение: заменить её на компактный wire-cube preview, а для PPM-текстур добавить реальное pixel-preview.
- **Визуальные артефакты docking resize.** Служебные окна resize-hitbox оставляли на границах layout тонкие рамки, хотя сами splitters уже были скрыты. Решение: отключить background и border у этих окон, оставив подсветку только при hover/drag.
- **Assert ImGui в окне viewport.** После фикса неподвижного viewport остался вызов SetCursorScreenPos() для отключённого drag-grip. ImGui считал, что код расширяет границы окна без последующего item, и в Debug-сборке вызывал assert. Решение: удалить мёртвый вызов и не создавать dock-controls для viewport.
- **Дрожание объектов при rotate/scale.** Первичная реализация гизмо каждый кадр раскладывала матрицу на position, rotation и scale и записывала все компоненты обратно. Из-за численного шума Euler-разложения scale мог менять rotation, а rotate мог менять scale. Решение: добавить безопасное разложение матрицы, unwrap углов, clamp scale и применять только тот компонент transform, который соответствует текущей операции гизмо.
- **Необходимость сохранения рабочего пространства.** После настройки панелей пользователь терял layout при перезапуске. Решение: сохранять

dock slots, visibility и размеры splitters в `assets/editor_layout.json`.

7. Ответы на контрольные вопросы

1. В чём отличие `immediate mode GUI` от `retained mode GUI`? Почему `ImGui` популярен в инструментах разработки игр?

В `immediate mode` интерфейс описывается заново каждый кадр, а состояние виджетов обычно хранится в коде приложения. В `retained mode` дерево виджетов создаётся заранее и живёт независимо. `ImGui` популярен в игровых инструментах, потому что он быстро интегрируется в `render loop`, не требует сложной архитектуры UI и удобен для отладочных и редакторских панелей.

2. Как организовать передачу событий ввода между `ImGui` и игровым движком?

Платформенные события сначала передаются `ImGui backend`, после чего игровой код проверяет флаги `io.WantCaptureMouse` и `io.WantCaptureKeyboard`. Если `ImGui` хочет захватить ввод, `gameplay` не должен реагировать на мышь или клавиатуру. Для редакторских `hotkeys` можно использовать отдельный `raw input path`, но с осторожной фильтрацией текстового ввода.

3. Как устроен `docking` в `ImGui`? Какие флаги необходимо установить для его поддержки?

В стандартной `docking`-ветке `ImGui` нужно включить флаг `ImGuiConfigFlags_DockingEnable` и создать `DockSpace`. В текущем проекте такая ветка недоступна, поэтому `docking` реализован вручную через логические слоты, `drag-and-drop payload` и позиционирование окон.

4. Каким образом изменения в инспекторе немедленно отражаются в окне предпросмотра?

Инспектор редактирует сами ECS-компоненты выбранной сущности. Рендерер каждый кадр читает эти компоненты, поэтому новая позиция, цвет, `mesh` или `collider` становятся видимыми без перезапуска сцены.

5. Как гизмо преобразует действия мыши в изменения матрицы трансформации?

ImGuizmo получает view/projection матрицы камеры, прямоугольник viewport и матрицу объекта. По движению мыши библиотека строит новую матрицу объекта. В проекте эта матрица проходит безопасное разложение, после чего в Transform записывается только релевантный компонент: position для перемещения, rotation для вращения или scale для масштабирования.

6. Какие данные необходимы для корректной работы 3D-манипулятора?

Нужны матрица вида камеры, матрица проекции, размеры viewport, мировая или локальная матрица объекта, выбранная операция transform и режим координат. Также желательно учитывать snapping и состояние selected entity.

7. Как можно оптимизировать отрисовку панели иерархии при большом количестве объектов?

Можно использовать фильтрацию, кэш отображаемых имён, ImGuiListClipper, ленивое раскрытие дерева и обновление списка только при изменениях ECS. Для тысяч объектов важно не строить лишние строки и не выполнять тяжёлые запросы каждый кадр.

8. Какие метрики производительности полезно отображать в панели статистики?

Полезны FPS, frame time, CPU/GPU render time, количество сущностей, renderable объектов, draw calls, активные rigidbodies, collision pairs, память ресурсов и время отдельных систем. В проекте уже отображаются FPS, frame ms, render ms и основные счётчики сцены.

9. Как организовать переключение между режимами редактирования и игры?

При входе в Play Mode редактор сохраняет snapshot сцены и включает физику/игровую логику. При выходе из Play Mode snapshot восстанавливается, чтобы пользователь вернулся к исходной редакторской сцене. В Edit Mode редактирование компонентов разрешено, а физика остано-

на.

8. Заключение

В ходе выполнения практической работы в *TurboGameEngine* был реализован WYSIWYG-редактор сцены на базе Dear ImGui и ImGuiZmo. Проект получил главное меню, профессионально оформленные панели, инспектор компонентов, иерархию сцены, viewport с live preview, манипуляторы трансформаций, статистику, режимы Edit/Play, выбор объектов кликом по сцене, Asset Browser, Undo/Redo и сохранение сцены. Дополнительно была разработана собственная docking-система с drag-and-drop, resize закреплённых областей и сохранением layout между запусками.

После первичной реализации редактор был доработан по результатам практического тестирования. Были исправлены скачки free camera, центральный viewport был зафиксирован как основная рабочая область, splitters перестали оставлять визуальные артефакты, был устранён debug assert ImGui, а операции rotate/scale в ImGuiZmo стали стабильнее за счёт безопасного разложения матрицы и покомпонентного применения transform.

Полученный редактор уже позволяет использовать движок не только как runtime-демонстрацию, но и как инструмент для интерактивной сборки и настройки сцен. Следующими логичными улучшениями являются полноценный material editor, drag-and-drop ассетов прямо во viewport, thumbnails через offscreen rendering, отдельные prefab-ресурсы, hot-reload ассетов и переход на ImGui docking branch при обновлении third-party зависимостей.

Репозиторий проекта на GitHub

<https://github.com/TurbikXD/TurboGameEngine>.